Capstone Technical Design and Implementation Baseline

Project Title: Code Escape

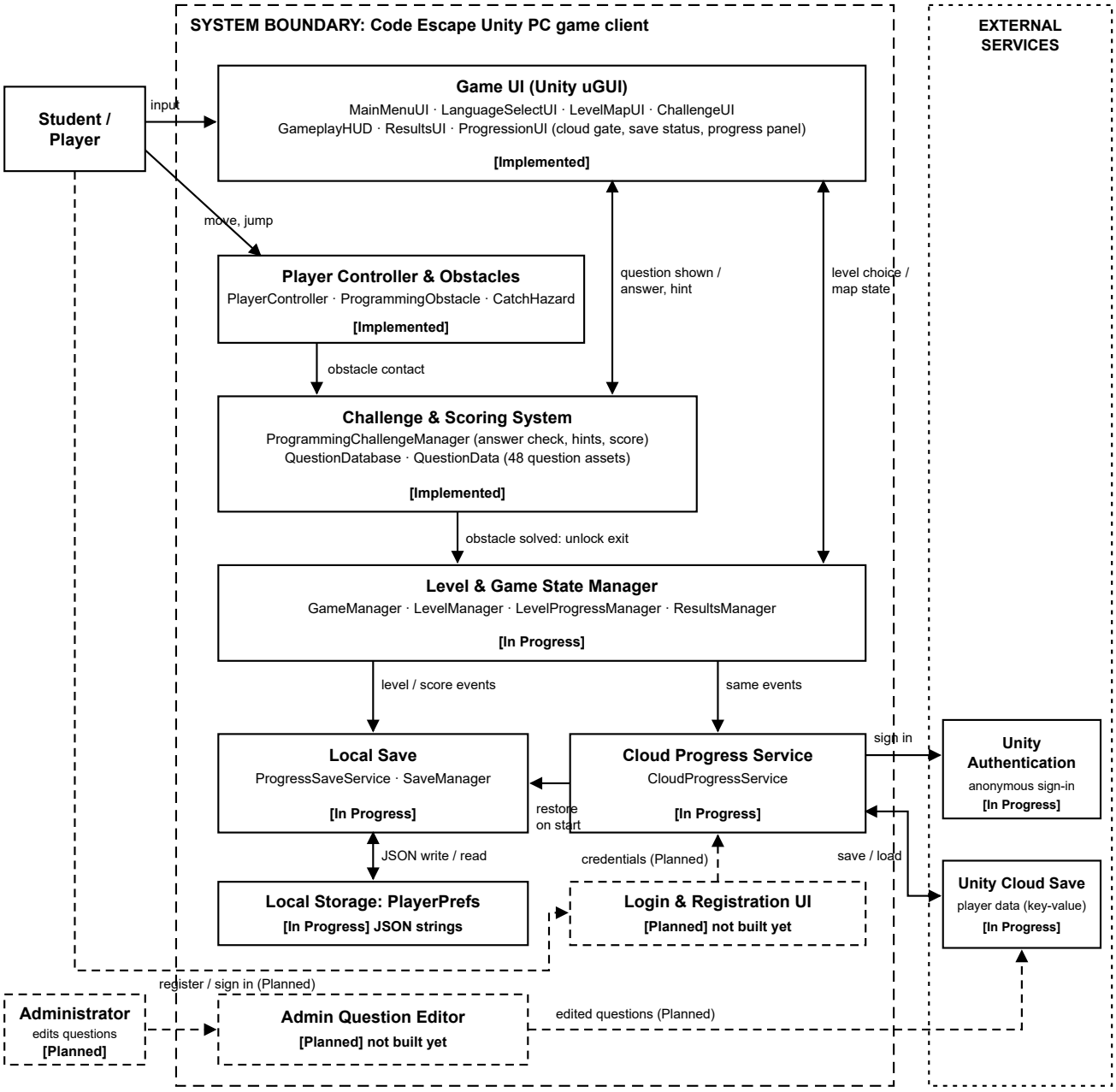
Educational programming puzzle game (Unity, PC)

Group Number	*[Group number]*
Group Members	*[Complete list of group members]*
Course / Section	*[Course and section]*
Instructor	*[Instructor]*
Date prepared	11 October 2026

Implementation status in this document reflects the project files inspected and the tests run on 11 October 2026.

Part I. Final System Architecture

Code Escape is a Unity PC game. Gameplay runs entirely in the game client. Progress is saved locally and copied to Unity Cloud Save, which is the only external service. The group does not use its own backend, API, SQL or Firebase database. A **Login and Registration system is planned and will be implemented soon**; it is not implemented yet, and the game currently signs the player in anonymously with no login screen. An **administrator account that can change the coding questions is also planned**; it depends on the login system and is not implemented.



Architecture explanation

Overall structure. A single Unity PC client contains the user interface, the game logic and the persistence layer. Unity Authentication and Unity Cloud Save are the only external services. The legend: the dashed rectangle is the system boundary, solid arrows are communication or data movement, and each box shows its implementation status.

User interaction. The student uses the keyboard and mouse on the Game UI (menus, level map, question panel) and moves the character with the Player Controller.

Main processing. It happens inside the client: obstacle contact opens a question, `ProgrammingChallengeManager` compares the answer with the stored solution (player code is never executed), updates the score, and the level managers unlock levels.

Persistent data. Progress is written to local storage (PlayerPrefs, JSON) and copied to Unity Cloud Save under the signed-in player. On start, the cloud copy is loaded and restored into the local save before gameplay is allowed.

Communication. Inside the client, components communicate through C# static events (`ChallengeCompleted`, `ScoreChanged`, `LevelUnlocked`, `LevelCleared`) and singleton `Instance` references. The client talks to the two Unity services through the Unity Gaming Services SDK over the internet.

External services. Unity Authentication (currently anonymous sign-in) and Unity Cloud Save (player-scoped key-value data). Not used: own backend or API, SQL or Firebase database. Planned: a Login and Registration UI that will feed the existing sign-in step, and an Admin Question Editor used by an administrator account to change the coding questions (both dashed in the diagram). Where edited questions will be stored is not decided yet; Unity Cloud Save Game Data (server-written custom items) is a candidate, and the game would then load questions from it instead of only from the bundled assets.

Changes from the previous alignment report

- Unity Authentication (anonymous) and Unity Cloud Save moved from Planned to Implemented (code present, tested in the Editor on 11 October 2026).
- A Login and Registration UI stays in the design as **Planned**: it will be implemented soon. Until then the game signs in anonymously, so progress is tied to the device and cannot be recovered on another one.
- Aligned with the Capstone System Cohesion Check: persistent progress is In Progress (local and cloud save implemented, login accounts planned); the cloud status UI is In Progress; the login system and the administrator account are Planned; the next milestone is the Login and Registration system.
- An administrator account with an Admin Question Editor is added as **Planned**. Questions are currently fixed ScriptableObject assets, so they cannot be changed without editing the project.

Part II. Component Responsibility Specification

Component	Responsibility	Input	Processing / Function	Output	Depends On
Game UI	Shows menus, level map, questions, score, results and cloud status	Mouse and keyboard; data from managers and the cloud service	Displays state, sends the player's choices and answers to the managers	Screens; level choice; submitted answer	Unity uGUI, Challenge & Scoring, Level & Game State, Cloud Progress Service
Login & Registration UI (Planned)	Lets a student register and sign in	Username and password (or chosen sign-in method)	Validates the input and passes it to the Cloud Progress Service, which calls Unity Authentication	Signed-in Player ID; error messages	Cloud Progress Service, Unity Authentication
Administrator account and Admin Question Editor (Planned)	Lets an administrator add, edit and remove coding questions	Administrator sign-in; question fields (text, code, choices, correct answer, hint, language, difficulty)	Checks the administrator role, validates the question, writes it to the question store	Updated question set; validation errors	Login system, question store (to be decided), Unity Authentication
Player Controller	Moves the character	Keyboard (A/D, arrows, W, Space)	Converts input into velocity (side-view or top-down mode)	Updated player position and animation state	Unity Input System, Rigidbody2D
Obstacles and Hazards	Block the way until a question is solved; catch the player	Player contact; solved flag	Opens a challenge on contact; unlocks the obstacle when solved	Challenge request; unlocked obstacle	Player Controller, Challenge & Scoring
Challenge & Scoring System	Runs a question, checks the answer, keeps the score	Challenge request; chosen or typed answer; hint request	Picks a question for the selected language, compares with the stored answer, adds 100 points per solved challenge, subtracts 25 for a hint	Correct or incorrect result; score; solved-obstacle event	Question Database, Game UI
Question Database	Holds the predefined questions	Language, topic, difficulty	Looks up a matching question asset	QuestionData	ScriptableObject assets
Level & Game State Manager	Tracks game state, unlocked and cleared levels and the story	Solved-obstacle events; level exit use; level choice	Unlocks the next level after required challenges, records cleared levels once, loads scenes	Unlocked/cleared levels; scene change; level-cleared event	Challenge & Scoring, Unity SceneManager
Local Save	Saves and loads the game state on the device	Game events; app pause and quit; restore request	Captures state from the managers and writes it as JSON (autosave); reads it back on start	Saved JSON; restored state	Local Storage, Level & Game State, Challenge & Scoring

Component	Responsibility	Input	Processing / Function	Output	Depends On
Cloud Progress Service	Signs in, loads and saves progress in the cloud, blocks play until loaded	Level and score events; Unity services responses	Initialises services, signs in anonymously, loads or creates the record, restores it, saves changes (batched), reports status	Saved/loaded cloud record; status; retry	Unity Authentication, Unity Cloud Save, Local Save
Local Storage	Stores persistent data on the device	JSON strings and settings values	Key-value storage (PlayerPrefs)	Stored / retrieved values	Unity PlayerPrefs
Unity Authentication	Gives the player an identity	Anonymous sign-in request	Creates or reuses an anonymous player session	Player ID and session token	Unity Gaming Services, internet
Unity Cloud Save	Stores player data online	Key-value pairs for the signed-in player	Stores and retrieves per-player data	Saved / retrieved items	Unity Authentication, internet

Part III. Data / Database Design

The project has no SQL or document database. Data is stored in two places: **local storage** (Unity PlayerPrefs, JSON strings) and **Unity Cloud Save** (player-scoped key-value items). Both store JSON text written with Unity's `JsonUtility`.

Storage structure

```
LOCAL STORAGE (PlayerPrefs, per device)
  CodeEscape.SelectedLanguage        int
  CodeEscape.Progress                JSON  ProgressData
  CodeEscape.LevelProgress.python    JSON  LevelProgressData
  CodeEscape.LevelProgress.php       JSON  LevelProgressData
  CodeEscape.LevelProgress.sql       JSON  LevelProgressData
  CodeEscape.MusicVolume, SfxVolume, Fullscreen, ResolutionWidth/Height
  CodeEscape.SaveVersion             int
  CodeEscape.Cloud.Unsynced, CodeEscape.Cloud.PlayerId   (cloud bookkeeping)

UNITY CLOUD SAVE (per authenticated player, default access class)
  code_escape_progression            JSON  PlayerProgressData
  code_escape_snapshot               JSON  ProgressSnapshot
                                           |-- progress_json     = ProgressData as text
                                           '-- level_progress[]  = { language, json =
LevelProgressData as text }
```

The cloud record is identified by the authenticated Unity Player ID: Cloud Save stores data under that player, so no shared key exists. Question content is not stored per player: it is 48 read-only ScriptableObject assets in `Assets/ScriptableObjects/Questions`.

Data dictionary

Entity / Collection	Field	Data Type	Key / Identifier	Purpose
Administrator role (Planned)	player_id, role	string, string	Unity Player ID	Marks which accounts may change questions. Must be checked on the server side, not only in the game.
Student account (Planned)	player_id, username	string	Unity Player ID	Would identify a registered student. Credentials are handled by Unity Authentication, not stored by the game.
PlayerProgressData (cloud)	player_id	string	Unity Player ID (owner)	Player who owns the record
	version	int		Record format version
	current_level	string (levelId)	References a level	Level currently reached
	highest_unlocked_level	string (levelId)	References a level	Furthest level unlocked in any language
	completed_levels	list of string	levelIds	Levels cleared, no duplicates

Entity / Collection	Field	Data Type	Key / Identifier	Purpose
	total_score	int		Lifetime score
	achievements	list of string	achievement ids	Unlocked achievements
	last_played	string (ISO 8601 UTC)		Time of latest progress update
	scored_challenges	list of string	"language:obstacleId"	Makes score counting once-only
	penalised_hints	list of string	"language:questionId"	Makes hint penalty counting once-only
ProgressSnapshot (cloud)	version	int		Snapshot format version
	language	int	1 Python, 2 PHP, 3 SQL	Selected language
	progress_json	string (JSON)		Playthrough state (ProgressData); empty if none active
	level_progress[].language / .json	int / string (JSON)	language	Level map progress of each language
ProgressData (local, inside snapshot)	version, language	int, int		Format version; language of the playthrough
	storyStage	int		Position in the story
	currentLevel	string (scene name)		Scene the player is in
	activeLevel	string (levelId)		Level started from the level map
	currentObjective	string		Objective text shown to the player
	completedEvents, unlockedLevels, completedLevels, completedChallenges, completedQuestions, hintedQuestions, collectedClues	list of string	ids	What the player has done, unlocked, solved, hinted and found
	score, hintsUsed	int		Current playthrough score and hints
	timePlayed	float		Seconds played
LevelProgressData (local, one per language)	language	string	"python", "php", "sql"	Language this progress belongs to
	completedLevels	list of string	levelIds	Cleared levels (unlocks the next one)
	lastPlayedLevel	string	levelId	Last level started

Entity / Collection	Field	Data Type	Key / Identifier	Purpose
	checkpoints[].levelId / .state	string / ProgressData	levelId	State each unlocked level starts from
SettingsData (local)	musicVolume, sfxVolume	float 0 to 1		Audio levels
	fullscreen, resolutionWidth, resolutionHeight	bool, int, int		Display preferences (0 x 0 = keep current)
QuestionData (asset, read-only)	id	string	Unique question id	Identifies the question
	language, topic, difficulty, questionType	enum, string, enum, enum		Selection and display
	questionText, code, choices, correctChoiceIndex, acceptedAnswers, hint, explanation	string, string, string[], int, string[], string, string		Question content and stored solution

Data integrity and validation

Rule	How it is enforced
A completed level appears only once	`LevelProgressData.MarkCompleted` and the cloud record use unique sets; `Sanitize` removes duplicates.
Level IDs must refer to existing levels	`LevelProgressData.Sanitize` drops unknown or empty IDs on load and before saving.
Score cannot be negative; counting twice is prevented	Score is clamped at 0 (`Mathf.Max`) and computed from distinct solved and hinted IDs, so repeated events do not add points.
The record belongs to the signed-in player	`player_id` is set from the authenticated Player ID; Cloud Save stores data per player.
Both cloud keys must exist together; data must be readable and not from a newer version	If only one key exists, JSON cannot be read, or the version is newer, the load fails with an error and nothing is overwritten.
Null or missing lists are repaired	`PlayerProgressData.Sanitize` replaces null fields and lists.
Language must be valid (Python, PHP, SQL)	`ProgrammingLanguage.IsValid` is checked before saving or loading progress by language.
Volumes stay between 0 and 1; resolution is both set or both 0	`SettingsData.Sanitize`.
Only an administrator may change questions (Planned)	Not implemented. The role must be checked by a server-side service, because the game client cannot be trusted.
An edited question stays valid (Planned)	Not implemented. Rules to apply: unique question id, valid language, at least two choices with a valid correct index (or an accepted answer for typed questions).
Unchanged data is not re-saved	A fingerprint of the record is compared with the last confirmed save before any cloud request.

Part IV. End-to-End Integration Design

Process 1: Save player progress

```
Player clears a level
 -> LevelManager / LevelProgressManager.RecordLevelCleared
 -> Local Save (level progress and playthrough written to PlayerPrefs)
 -> LevelCleared event -> Cloud Progress Service (RequestSave)
 -> Build record (PlayerProgressData + ProgressSnapshot), compare with last confirmed
 -> Unity Cloud Save (SaveAsync, two keys)
 -> Save confirmed
 -> Game UI: "Saved to cloud"
```

Item	Description
Trigger	A level is cleared. Also: level unlocked, score changed, challenge solved (batched after 3 seconds), main menu loaded, app paused.
Source Component	Level & Game State Manager (and Challenge & Scoring for score events)
Data Generated / Sent	`PlayerProgressData` JSON and `ProgressSnapshot` JSON (two Cloud Save keys)
Processing Component	Cloud Progress Service (builds the record, removes duplicates, computes the score, unlocks achievements)
Destination	Unity Cloud Save, under the signed-in player. The local save is written first.
Stored / Retrieved Data	Stored: current and highest unlocked level, completed levels, total score, achievements, timestamp, restore snapshot
Expected Response	Cloud Save confirms the write; the service marks the record as confirmed and clears the "unsynced" flag.
User Output	Status chip: "Saving to cloud...", then "Saved to cloud" with the time. "Saved" is shown only after confirmation.
If it fails	**No internet or save error / timeout (20 s):** the chip shows "Cloud save failed" with the reason and a Retry button; progress stays in the local save and is flagged unsynced; the game keeps running. Retry sends the full current state. On the next launch, unsynced local changes of the same player are sent again instead of being overwritten. **Data unchanged:** no request is sent. **Invalid input:** unknown level IDs are dropped and negative scores clamped before sending.

Planned change: when the Login and Registration UI is implemented, the sign-in step in Process 2 will use the student's registered account instead of an anonymous session. The rest of both flows stays the same.

Process 2: Load saved progress at startup

```
Game starts
 -> Cloud Progress Service
 -> Unity Gaming Services initialised
 -> Unity Authentication (anonymous sign-in) -> Player ID
 -> Unity Cloud Save (LoadAsync, two keys)
 -> Record found: restore into Local Save, managers, language and level map
    Record missing: create a new record from the current state and save it
 -> Game UI: gate removed, level map and progress panel show the data
```

Item	Description
Trigger	The game opens (first scene loaded), or the player presses Retry on the connection screen.
Source Component	Cloud Progress Service
Data Generated / Sent	Anonymous sign-in request; load request for `code_escape_progression` and `code_escape_snapshot`
Processing Component	Unity Authentication, Unity Cloud Save, then Cloud Progress Service and Local Save (restore)
Destination	Local Save, Level & Game State Manager, Language Manager, Game UI
Stored / Retrieved Data	Retrieved: the saved record and snapshot. Written locally: language, playthrough, level progress per language.
Expected Response	Player ID obtained; record loaded and applied; state becomes Ready.
User Output	"Connecting to Unity Cloud Save" screen, then the main menu. The level map shows saved unlocks and completions, and the Cloud progress panel shows the cloud data.
If it fails	**No internet or timeout:** a full-screen "Cannot connect to Cloud Save" message with the reason and a Retry button; gameplay stays locked. Nothing is reset. **Record not found (new player):** a new record is created and saved; the game unlocks only after the save is confirmed. **Only one key found, data unreadable, or newer version:** an error is shown, the stored data is left untouched, and the player cannot start until it is resolved. **Project not linked or service disabled:** initialisation error shown on the same screen.

Part V. Repository and Implementation Structure

Repository name	IT22_DatabaseConnectivity (GitHub, created and managed with GitHub Desktop)
Repository link	https://github.com/jjcmall/IT22_DatabaseConnectivity
Repository visibility	Private *[confirm; add the instructor as collaborator]*
Development platform	Unity 6000.5.10f1, C#, Universal Render Pipeline (2D / 2.5D), Input System, uGUI
README status	Exists: `README.md` (overview, controls, requirements, structure) and `Documentation/CloudSave.md` (Cloud Save details)
.gitignore status	Exists (Unity template), so Library, Temp, Logs and UserSettings are excluded
Git LFS	Not configured (`.gitattributes` has no LFS rules). The largest asset is about 33 MB (`Assets/Animations/Sam/Sam.fbx`), below GitHub's 100 MB file limit, so LFS is not required now. Reconsider if larger assets are added.
Branching approach	Branch `main` exists. Further branching approach: *[to be agreed by the group]*

Main folders

```
Code Escape/
|-- Assets/
|   |-- Scenes/                00_MainMenu ... 10_Results (12 scenes)
|   |-- Scripts/
|   |   |-- Player/            PlayerController, cameras
|   |   |-- Obstacles/         ProgrammingObstacle, CatchHazard, HazardMotion
|   |   |-- Programming/       ProgrammingChallengeManager, QuestionData, QuestionDatabase,
LanguageManager
|   |   |-- Game/              GameManager, LevelManager, LevelProgressManager, ResultsManager,
SceneTransitionManager
|   |   |-- SaveSystem/        SaveManager, ProgressSaveService, ProgressData, LevelProgressData,
SettingsData
|   |   |-- Cloud/             CloudProgressService, PlayerProgressData, ProgressionUI
|   |   |-- UI/                MainMenuUI, LevelMapUI, ChallengeUI, GameplayHUD, ResultsUI, ...
|   |   |-- Story/ Clues/ Dialogue/ Cutscenes/ Audio/
|   |-- ScriptableObjects/Questions/   Python, PHP, SQL, Hazards
|   |-- Prefabs/Resources/GameSystems.prefab
|   |-- Art/ Art3D/ Sprites/ Animations/ Audio/ Fonts/ UI/
|-- Packages/manifest.json
|-- ProjectSettings/
|-- Documentation/
|-- README.md
```

Where the architecture components are located

Architecture Component	Repository / Project Location
Admin Question Editor (Planned)	Not created yet; questions currently live in `Assets/ScriptableObjects/Questions`
Login & Registration UI (Planned)	Not created yet; expected in `Assets/Scripts/UI` with a new scene in `Assets/Scenes`

Architecture Component	Repository / Project Location
Game UI	`Assets/Scripts/UI`; scenes in `Assets/Scenes`; cloud gate and status in `Assets/Scripts/Cloud/ProgressionUI.cs`
Player Controller	`Assets/Scripts/Player/PlayerController.cs`
Obstacles and Hazards	`Assets/Scripts/Obstacles`
Challenge & Scoring System	`Assets/Scripts/Programming/ProgrammingChallengeManager.cs`
Question Database	`Assets/Scripts/Programming/QuestionDatabase.cs`, `QuestionData.cs`; assets in `Assets/ScriptableObjects/Questions`
Level & Game State Manager	`Assets/Scripts/Game` (LevelManager, LevelProgressManager, GameManager)
Local Save and Local Storage	`Assets/Scripts/SaveSystem`
Cloud Progress Service and data classes	`Assets/Scripts/Cloud`
Unity Authentication, Cloud Save (packages)	`Packages/manifest.json` (`com.unity.services.cloudsave` 3.4.1; Authentication 3.7.4 and Core 1.18.0 as dependencies)
Persistent global managers	`Assets/Prefabs/Resources/GameSystems.prefab`, created by `GameBootstrapper`

Part VI. Implementation Baseline and Development Plan

Evidence was taken from the project files and from Editor Play Mode tests on 11 October 2026. "Implemented" means the code or asset exists; features not tested in a real play-through are noted. There is no commit history to cite yet.

Component / Feature	Current State	Evidence	Next Action	Assigned Member/s
Player movement	Implemented	`PlayerController.cs`	Test on a built game	*[Member]*
Obstacles and hazards	Implemented	`ProgrammingObstacle.cs`, `CatchHazard.cs`; nine obstacle IDs in scenes 03 to 08	Play-test every obstacle	*[Member]*
Questions and scoring	Implemented	`ProgrammingChallengeManager.cs` (100 / -25); 48 question assets	Review question content; test scoring by hand	*[Member]*
Level progression and level map	Implemented	`LevelManager.cs`, `LevelProgressManager.cs`, `01b_LevelMap.unity`; Editor test showed Home completed, Neighborhood available, Warehouse locked after reload	Test all six levels	*[Member]*
Results screen	Implemented	`ResultsManager.cs`, `10_Results.unity`	Decide whether finished results must be kept (currently the local save is cleared at the end)	*[Member]*
Local save	Implemented	`SaveManager.cs`, `ProgressSaveService.cs`	Include in automated tests	*[Member]*
Anonymous authentication	Implemented	`CloudProgressService.cs`; Editor Play Mode obtained a Player ID without a login UI	Test in a standalone build	*[Member]*
Cloud Save load / save / restore	Implemented	`CloudProgressService.cs`; Editor test: record created, saved after a level clear without duplicates, loaded on restart; data seen in the Unity Dashboard	Test network failure and retry; test quit with unsaved changes	*[Member]*
Cloud status UI	In Progress	`ProgressionUI.cs` compiles; not yet seen on screen	Check layout in the Game view and fix overlaps	*[Member]*
Achievements	In Progress	Five derived achievements in `CloudProgressService.cs`; only `first_level` triggered in tests	Confirm the list with the group; test all	*[Member]*
Login and registration system	Planned	None yet. The game uses anonymous sign-in, which cannot recover progress on another device.	Build the Login and Registration UI (milestone 1)	*[Member]*
Administrator account and question editing	Planned	None. Questions are ScriptableObject assets in `Assets/ScriptableObjects/Questions`.	Decide the question store and role check; build after the login system (milestone 1)	*[Member]*

Component / Feature	Current State	Evidence	Next Action	Assigned Member/s
Automated tests	Planned	Unity Test Framework installed; no test scripts found	See milestone 3	*[Member]*
Repository	In Progress	GitHub repository IT22_DatabaseConnectivity on `main` with Unity `.gitignore`; project files and commits being pushed	Push the project files with clear commits; add commit hashes to this document	*[Member]*

Administrator question editing depends on the login system (milestone 1) and a decision on where edited questions are stored, so it follows milestone 1 and is not one of the next three milestones.

Next three technical milestones

Order	Technical Milestone	Dependency / What Must Exist First	Expected Output
1	Implement the Login and Registration system	Cloud save and the sign-in step (exist); a sign-in method chosen from those Unity Authentication supports; a Login scene design	A Login and Registration UI connected to Unity Authentication, replacing anonymous sign-in; progress tied to the registered account and retrievable on another device; updated diagram and data design
2	Verify and harden cloud save and load with accounts, including failure cases and a standalone build	Milestone 1; a Windows build; a way to disable the network	Written test results; fixes for any defects found; confirmed gate, retry and unsynced-change behaviour
3	Add automated tests for progress rules	Stable data model (Part III); Unity Test Framework (installed)	Edit Mode tests for duplicate-level prevention, score counting, sanitising and record building; a passing test run

References

- Unity Technologies. Unity Cloud Save documentation. docs.unity.com (Unity Gaming Services, Cloud Save).
- Unity Technologies. Unity Authentication documentation (anonymous sign-in). docs.unity.com.
- Unity Technologies. Unity Manual: PlayerPrefs and JsonUtility. docs.unity.com.
- Project files: `README.md` and `Documentation/CloudSave.md` in the Code Escape repository.